

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AI & ML

LABORATORY OBSERVATION RECORD

DIGITAL PAYMENT – OBJECT ORIENTED PROGRAMMING IN JAVA

1. TITLE OF THE EXPERIMENT

Implementation of a Digital Payment System using Interface, Encapsulation and Custom Exception Handling in Java

2. AIM

To design and implement a Java-based digital payment system that simulates wallet creation, money addition, UPI payment, balance checking and transaction validation using interfaces, encapsulation, String operations, custom exceptions and try-catch-finally exception handling.

3. ALGORITHM

Step 1: Start the program.

Step 2: Define the PaymentService interface and declare pay() and checkBalance() methods.

Step 3: Define the custom exceptions InvalidUPIException, InvalidAmountException and InsufficientBalanceException by extending Exception.

Step 4: Define the Wallet class with private data members userName, mobileNumber, upild and balance to demonstrate encapsulation.

Step 5: Create the Wallet constructor and initialize the user's wallet details.

Step 6: Implement addMoney() to validate the amount and add valid money to the wallet.

Step 7: Implement getBalance(), getUpild() and displayWalletDetails() to access and display wallet information.

Step 8: Define the UPIPayment class and implement the PaymentService interface.

Step 9: Associate the UPIPayment object with the Wallet object using its constructor.

Step 10: Validate the receiver UPI ID using String operations such as contains(), startsWith() and endsWith().

Step 11: If the UPI ID is invalid, throw InvalidUPIException.

Step 12: Validate the payment amount. If the amount is zero or negative, throw InvalidAmountException.

Step 13: Compare the payment amount with the wallet balance. If the balance is insufficient, throw InsufficientBalanceException.

Step 14: If all validations are successful, deduct the payment amount from the wallet and display the payment success message.

Step 15: Use try-catch-finally blocks to handle transaction exceptions.

Step 16: Display the final wallet balance and wallet details, then terminate the program.

4. EXECUTION STEPS

Step 1: Compile the Java source file with package name task and public class name DigitalPayment.

Step 2: Run the DigitalPayment program.

Step 3: Observe the initial wallet details displayed for Rahul.

Step 4: Verify that Rs.2000 is added successfully and the balance becomes Rs.7000.

Step 5: Observe the UPI payment to arun@upi for Rs.1500.

Step 6: Verify the UPI validation, amount validation and sufficient-balance validation.

Step 7: Verify that Rs.1500 is deducted and the final balance becomes Rs.5500.

Step 8: Observe the transaction completion message produced by the finally block.

Step 9: Compare the displayed output with the expected output.

5. SOURCE CODE

```
package task;

//1. PaymentService Interface
interface PaymentService {

    void pay(String receiverUPI, double amount)
        throws InvalidUPIException, InvalidAmountException, InsufficientBalanceException;

    void checkBalance();
}

//2. Custom Exception - Invalid UPI
class InvalidUPIException extends Exception {

    public InvalidUPIException(String message) {
        super(message);
    }
}

//3. Custom Exception - Invalid Amount
class InvalidAmountException extends Exception {

    public InvalidAmountException(String message) {
        super(message);
    }
}

//4. Custom Exception - Insufficient Balance
class InsufficientBalanceException extends Exception {

    public InsufficientBalanceException(String message) {
        super(message);
    }
}

//5. Wallet Class
class Wallet {

    public void deductMoney(double amount) {
        balance = balance - amount;
    }

    // Encapsulation - private data members
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    // Constructor
    public Wallet(String userName, String mobileNumber, String upiId, double balance) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = balance;
    }

    // Add money
    public void addMoney(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                "Invalid amount. Amount must be greater than zero.");
        }

        balance = balance + amount;
        System.out.println("Money added successfully: Rs." + amount);
    }

    // Get balance
    public double getBalance() {
        return balance;
    }

    // Get UPI ID
    public String getUpiId() {
        return upiId;
    }

    // Display wallet details
    public void displayWalletDetails() {
        System.out.println("\n----- Wallet Details -----");
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.println("Balance      : Rs." + balance);
    }
}
```

```

    }
}

//6. UPIPayment Class
class UPIPayment implements PaymentService {

    private Wallet wallet;

    // Constructor
    public UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    // Payment method
    @Override
    public void pay(String receiverUPI, double amount)
        throws InvalidUPIException,
        InvalidAmountException,
        InsufficientBalanceException {

        // Validate UPI ID using String operations
        if (receiverUPI == null ||
            !receiverUPI.contains("@") ||
            receiverUPI.startsWith("@") ||
            receiverUPI.endsWith("@")) {

            throw new InvalidUPIException(
                "Invalid UPI ID: " + receiverUPI);
        }

        // Validate amount
        if (amount <= 0) {
            throw new InvalidAmountException(
                "Invalid payment amount. Amount must be greater than zero.");
        }

        // Check balance
        if (amount > wallet.getBalance()) {
            throw new InsufficientBalanceException(
                "Insufficient balance. Available balance: Rs."
                + wallet.getBalance());
        }

        // Deduct amount
        wallet.deductMoney(amount);

        System.out.println("\nPayment successful!");
        System.out.println("Receiver UPI : " + receiverUPI);
        System.out.println("Amount paid : Rs." + amount);
    }

    // Check balance
    @Override
    public void checkBalance() {
        System.out.println("\nAvailable Balance: Rs." + wallet.getBalance());
    }
}

public class DigitalPayment {

    public static void main(String[] args) {

        // Create wallet
        Wallet wallet = new Wallet(
            "Rahul",
            "9876543210",
            "rahul@upi",
            5000
        );

        // Create UPIPayment object
        UPIPayment payment = new UPIPayment(wallet);

        // Display initial wallet details
        wallet.displayWalletDetails();

        // Add money
        try {
            wallet.addMoney(2000);
        }
        catch (InvalidAmountException e) {
            System.out.println("Error: " + e.getMessage());
        }

        payment.checkBalance();
        // Make payment
    }
}

```

```

    try {
        payment.pay("arun@upi", 1500);
        System.out.println("Transaction completed successfully.");
    }
    catch (InvalidUPIException e) {
        System.out.println("UPI Error: " + e.getMessage());
    }
    catch (InvalidAmountException e) {
        System.out.println("Amount Error: " + e.getMessage());
    }
    catch (InsufficientBalanceException e) {
        System.out.println("Balance Error: " + e.getMessage());
    }
    finally {
        System.out.println("\nTransaction process completed.");
    }

    // Final balance
    payment.checkBalance();

    // Final wallet details
    wallet.displayWalletDetails();
}
}

```

6. OUTPUT / EXECUTION

```
----- Wallet Details -----
User Name      : Rahul
Mobile Number  : 9876543210
UPI ID        : rahul@upi
Balance       : Rs.5000.0
Money added successfully: Rs.2000.0

Available Balance: Rs.7000.0

Payment successful!
Receiver UPI : arun@upi
Amount paid  : Rs.1500.0
Transaction completed successfully.

Transaction process completed.

Available Balance: Rs.5500.0

----- Wallet Details -----
User Name      : Rahul
Mobile Number  : 9876543210
UPI ID        : rahul@upi
Balance       : Rs.5500.0
```

7. OUTPUT EXPLANATION

- Wallet Details: The Wallet object displays Rahul's name, mobile number, UPI ID and initial balance.
- Money Addition: addMoney(2000) validates the amount and increases the balance from Rs.5000.0 to Rs.7000.0.
- Balance Check: checkBalance() displays the available wallet balance.
- UPI Payment: The receiver UPI ID arun@upi passes the String validation and Rs.1500.0 is processed.
- Transaction Validation: The payment amount is positive and the wallet has sufficient balance, so no custom exception is thrown.
- Payment Success: deductMoney() reduces the wallet balance to Rs.5500.0 and the success message is displayed.
- Finally Block: The transaction process completion message is displayed by the finally block.
- Final Wallet Details: The final wallet balance is displayed as Rs.5500.0.

8. RESULT

The Digital Payment program was successfully implemented and executed. The program demonstrates interface implementation, encapsulation, object association, String-based UPI validation, custom exception handling, transaction validation, and try-catch-finally processing.

9. LEARNING OUTCOME

Understood how interfaces, classes, objects, private data members, constructors, String methods, custom exceptions, throw/throws, try-catch-finally and method overriding can be combined to implement a simple real-world digital payment application.